

UNIVERSIDAD RAFAEL LANDÍVAR

Facultad de Ingeniería — Ingeniería en Informática y Sistemas
Campus San Alberto Hurtado, S.J. de Quetzaltenango
Agosto de 2026

DOCUMENTO TÉCNICO

Laboratorio No. 1 — Manejo e Implementación de Archivos: Aplicación de Escritorio con Gestión de Configuración de Usuario

Jonathan André Soto Tzul
1533825

1. Introducción

El presente documento describe el diseño técnico e implementación de una aplicación de escritorio desarrollada en Python utilizando Tkinter para la interfaz gráfica. El objetivo principal del sistema es gestionar la persistencia y carga de la configuración de usuario (nombre, tema de interfaz, idioma, tamaño de fuente, colores de interfaz y foto de perfil) mediante un esquema propio de almacenamiento y manipulación de archivos. Se aplican técnicas de escritura atómica mediante archivos temporales, gestión de copias de respaldo (.bak), tolerancia a fallos mediante capturas de excepciones y conservación de codificación de caracteres.

2. Comparación y justificación del formato de almacenamiento

Para la persistencia de datos del sistema se evaluaron dos alternativas de formato de almacenamiento:

Criterio	Formato Seleccionado (Binario con struct)	Formato Descartado (Texto Plano / CSV)
Estructura de Datos	Empaquetado estricto por bytes secuenciales mediante struct (>H, >B, >i).	Formato plano delimitado por comas o saltos de línea.

Criterio	Formato Seleccionado (Binario con struct)	Formato Descartado (Texto Plano / CSV)
Eficiencia y Tamaño	Muy compacto; optimiza el espacio en disco al guardar enteros y colores en pocos bytes.	Mayor tamaño en disco al representar números y datos como cadenas de texto.
Seguridad de Datos	Dificulta la modificación accidental o no autorizada desde editores de texto externos.	Propenso a corrupción manual por eliminación de delimitadores o comas.
Parsing y Deserialización	Deserialización directa en memoria sin necesidad de parsear texto sintáctico.	Requiere convertir manualmente cadenas a enteros, tuplas y tipos nativos.

Justificación Técnica: Se eligió la serialización binaria propia para cumplir con los requisitos de la práctica de gestión de archivos. Esto permite empaquetar cadenas con su longitud explícita en 2 bytes (>H), enteros de 4 bytes (>i), valores de tema/booleano en 1 byte (>B) y tuplas de colores RGB (>BBB), logrando una lectura y escritura eficiente y directa.

3. Implementación de la escritura segura y respaldo (Backup)

Para prevenir la corrupción del archivo de configuración ante cierres abruptos o fallos del sistema durante la escritura, se implementó un mecanismo atómico compuesto por tres pasos esenciales:

1. **Creación de Respaldo (config.bin.bak):** Si el archivo principal config.bin ya existe en el disco, se genera una copia idéntica antes de realizar cambios usando `shutil.copy2`.
2. **Escritura Temporal (config.tmp):** Toda la información serializada se escribe primero en un archivo de paso temporal.
3. **Reemplazo Atómico:** Con `os.replace`, el archivo temporal reemplaza atómicamente a config.bin.

Código de implementación:

Python

```
def guardar_configuracion(config: dict) -> bool:
    try:
```

```

        os.makedirs(os.path.dirname(RUTA_CONFIG),
exist_ok=True)

# 1. Respaldo del archivo actual, si existe
if os.path.exists(RUTA_CONFIG):
    shutil.copy2(RUTA_CONFIG, RUTA_BACKUP)

# 2. Escritura completa a archivo temporal
datos = _serializar(config)
with open(RUTA_TMP, "wb") as archivo:
    archivo.write(datos)

# 3. Reemplazo atómico
os.replace(RUTA_TMP, RUTA_CONFIG)
return True

except PermissionError:
    print("Error: sin permisos para escribir el archivo
de configuración.")
    return False
except OSError as error:
    print(f"Error al guardar la configuración:
{error}")
    return False

```

4. Manejo de errores y tolerancia a fallos

La aplicación fue diseñada para degradar de forma controlada ante situaciones anómalas en el sistema de archivos, evitando que la aplicación sufra caídas inesperadas:

```

Python
def cargar_configuracion() -> dict:
    try:

```

```

with open(RUTA_CONFIG, "rb") as archivo:
    datos = archivo.read()
    return _deserializar(datos)

except FileNotFoundError:
    print("Aviso: no existe un archivo de configuración
previo. Usando valores por defecto.")
    return dict(Valores)

except PermissionError:
    print("Aviso: sin permisos para leer el archivo de
configuración. Usando valores por defecto.")
    return dict(Valores)

except (struct.error, UnicodeDecodeError, IndexError)
as error:
    print(f"Aviso: el archivo de configuración está
corrupto o tiene formato inválido ({error}). Usando valores
por defecto.")
    return dict(Valores)

```

5. Estructura de Clases y Arquitectura del Sistema

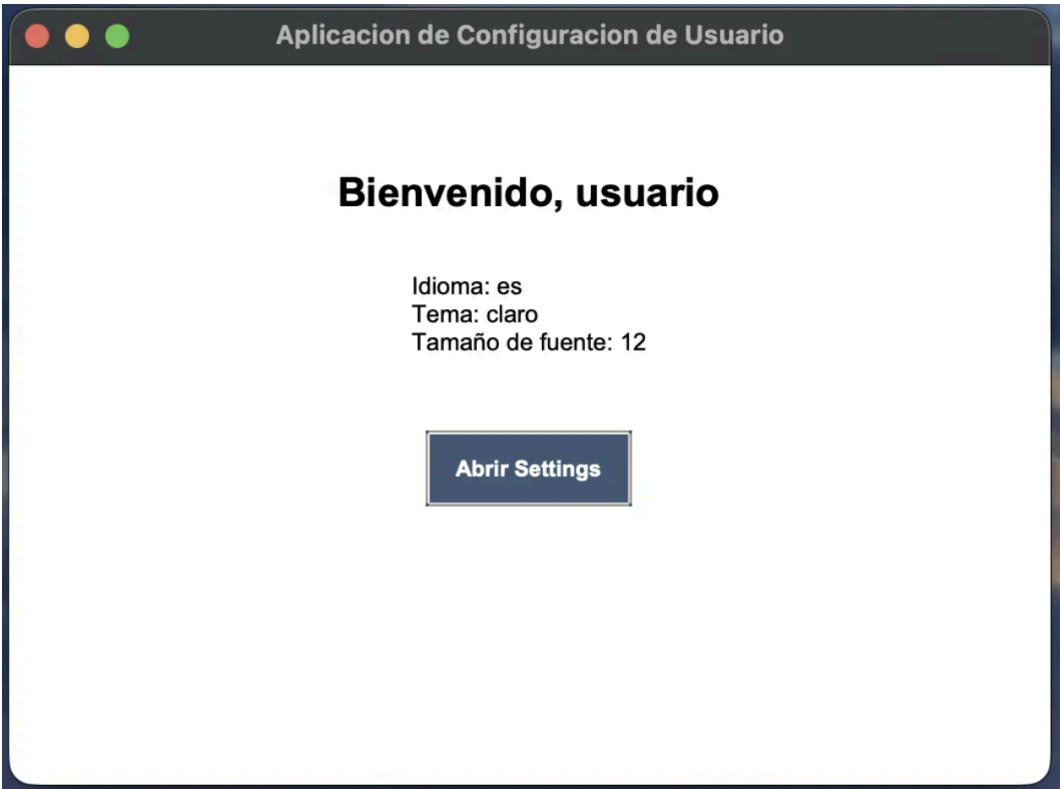
Módulo / Clase	Responsabilidad y Contenido
main.py	Punto de entrada del programa. Ejecuta la carga inicial de configuración y levanta la ventana principal.
core/config_manager.py	Contiene las funciones de serialización/deserialización binaria (_serializar, _deserializar), lectura y escritura atómica.
gui/main_window.py (MainWindow)	Ventana principal en Tkinter. Carga y aplica el tema visual (colores y tamaños de fuente) y despliega el menú de navegación.

Módulo / Clase	Responsabilidad y Contenido
gui/settings_window.py (SettingsWindow)	Ventana modal de ajustes. Proporciona formularios para modificar nombre, tema, idioma, colores y foto de perfil.

6. Evidencia del Proceso de Pruebas y Funcionamiento del Sistema

Etapa 1 — Ventana Principal en Estado Inicial (Tema Claro)

Muestra la interfaz de bienvenida inicial tras realizar la carga de los valores predeterminados del sistema: usuario "usuario", idioma "es", tema "claro" y tamaño de fuente 12. Muestra además la barra de menús simulada y el botón principal de acceso a Settings.



Etapa 2 — Formulario de Configuración (Ventana Settings)

Muestra la ventana emergente SettingsWindow. Permite al usuario modificar los parámetros de la aplicación: ingresar el nombre de usuario ("usuario"), seleccionar tema de interfaz (Claro/Oscuro), seleccionar el idioma (es, es-ES, en, en-US), especificar el tamaño de fuente, seleccionar colores mediante colorchooser y cargar la foto de perfil mediante el selector de archivos del sistema.

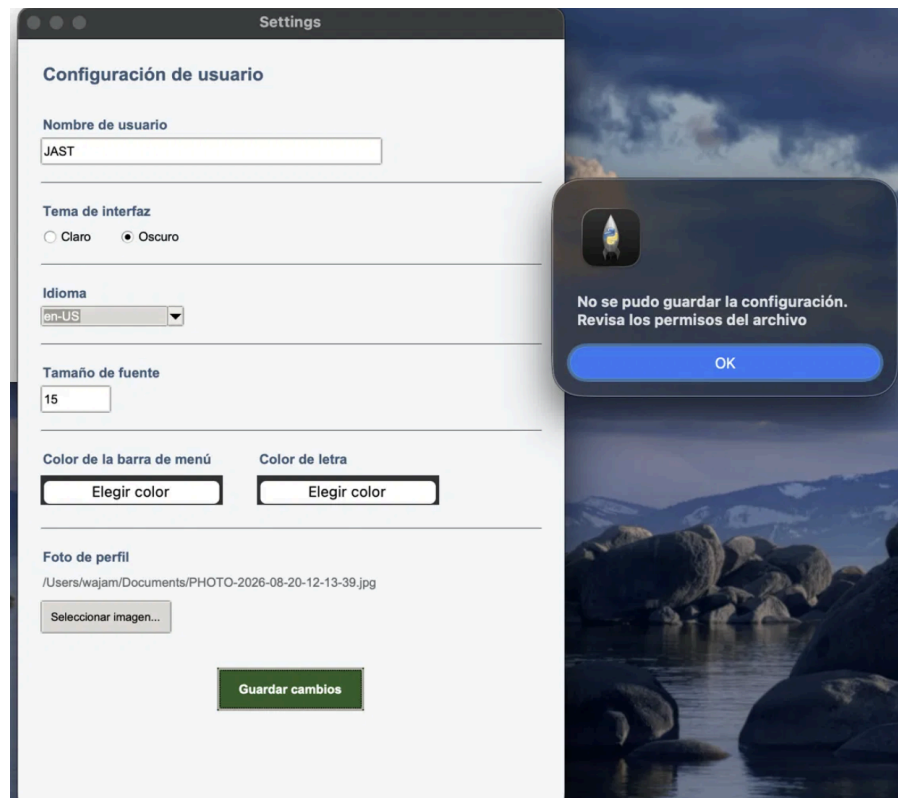
The image shows a macOS-style window titled "Settings" with a title bar containing red, yellow, and green window control buttons. The main content area is titled "Configuración de usuario" and contains several settings sections separated by horizontal lines:

- Nombre de usuario:** A text input field containing the text "usuario".
- Tema de interfaz:** Two radio buttons labeled "Claro" (selected) and "Oscuro".
- Idioma:** A dropdown menu showing "es".
- Tamaño de fuente:** A text input field containing the number "12".
- Color de la barra de menú:** A button labeled "Elegir color".
- Color de letra:** A button labeled "Elegir color".
- Foto de perfil:** The text "(ninguna seleccionada)" is displayed above a button labeled "Seleccionar imagen...".

At the bottom center of the window is a large green button labeled "Guardar cambios".

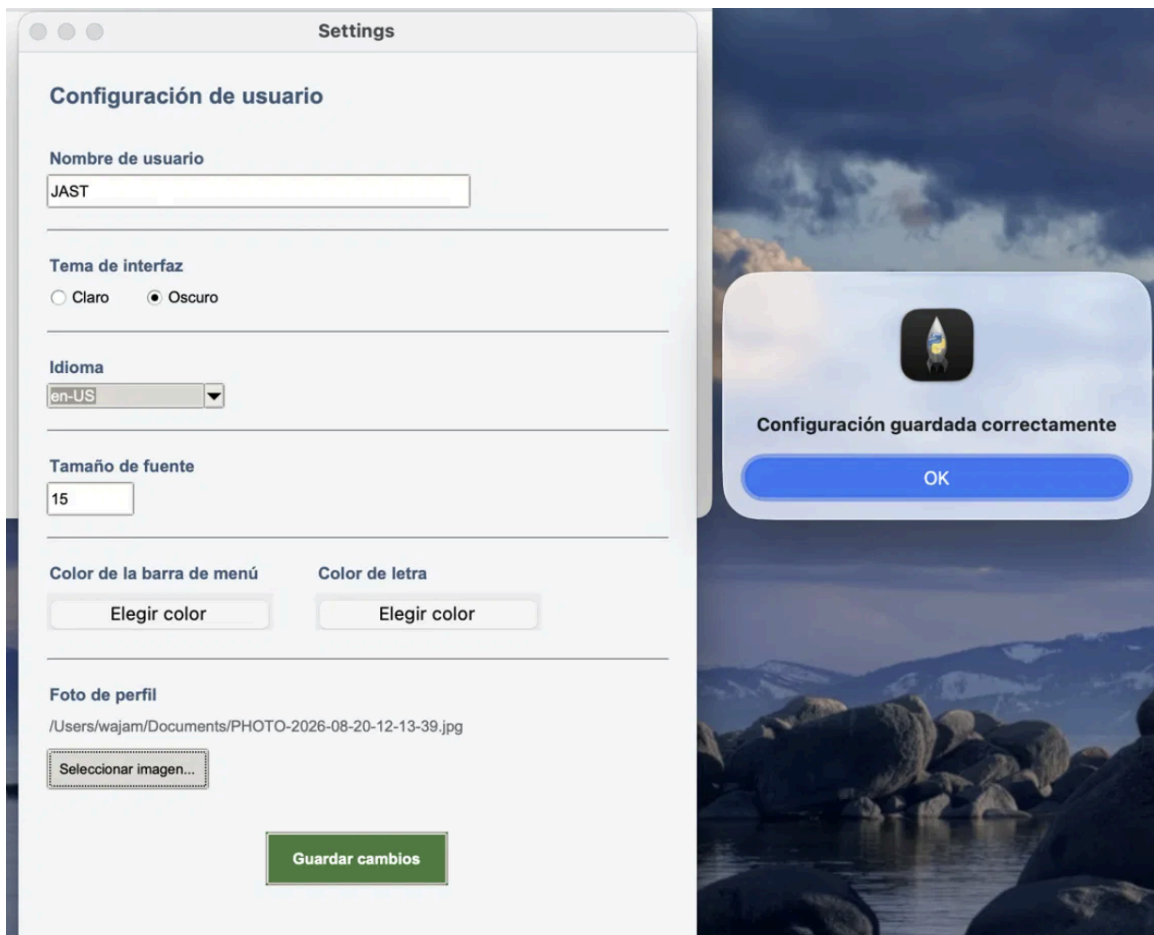
Etapas 3 — Prueba de Manejo de Errores (Sin Permisos de Escritura)

Se simula un caso donde el archivo o directorio carece de permisos de escritura. Al intentar presionar "Guardar cambios", la función `config_manager.guardar_configuracion()` captura la excepción `PermissionError` y retorna `False`. La interfaz responde mostrando un diálogo modal de error: "No se pudo guardar la configuración. Revisa los permisos del archivo".



Etapas 4 — Confirmación de Guardado Exitoso

Una vez corregidos los permisos, la aplicación ejecuta el proceso seguro de guardado (config.tmp -> config.bin.bak -> config.bin). La ventana confirma el éxito del proceso mediante el mensaje: "Configuración guardada correctamente".



Etapas 5 — Ventana Principal Actualizada Dinámicamente (Tema Oscuro)

Al cerrar la ventana de Settings, la ventana principal recibe los nuevos datos guardados a través del callback al `_guardar_callback` y vuelve a renderizar la interfaz invocando `_aplicar_configuracion_visual()`. En la pantalla se aprecia la actualización inmediata: fondo oscuro (#2b2b2b), texto de saludo "Bienvenido, JAST", idioma "en-US", tema "oscuro" y tamaño de fuente incrementado a 15.



7. Conclusiones

1. La implementación de serialización binaria manual mediante el módulo struct demuestra un manejo directo de bytes en el disco duro, optimizando el espacio de almacenamiento y la lectura de datos.
2. El flujo de guardado seguro con archivos temporales (.tmp), reemplazo atómico y copia de respaldo (.bak) garantiza que la configuración del usuario no se corrompa en caso de interrupciones de energía o cierres inesperados durante la escritura.